

Introducción al Curso

Joaquín Ballesteros. Grupo C.

Dpto. Lenguajes y Ciencias de la Computación.

Universidad de Málaga

¿Quién os da clase de teoría?

Joaquín Ballesteros

Ing. Informática; Doctor en Ing.
Telecomunicación.

Despacho: 3.3.2_III

Contacto:

- Utiliza siempre correo jballesteros@uma.es o nuestro [Servidor de Discord](#)



Equipo de prácticas

- Joaquín Ballesteros
- José Enrique Gallardo
- ¿?

Laboratorios: 3.1.9, 3.1.10

Horario

- *Lunes, 19:30-21:15.* Teoría en **2.0.8**
- *Miércoles, 17:30-19:15.* Teoría en **2.0.8**, Sesiones de laboratorio **laboratorio 3.1.6, 3.1.7, 3.1.9.**
- *Viernes, 15:30-17:15:* Teoría en **2.0.8** (11 sep, 18 sep, 9 oct, 27 nov)

Calendario

Hay un Calendario de Google con el horario en Campus Virtual con todos los detalles. Este calendario puede ser actualizado durante el curso, así que por favor revísalo regularmente.

[Link calendario](#)

Campus Virtual

- Toda la información sobre el curso estará disponible en Campus Virtual.
- Si no puedes entrar, envíame un correo y resolvemos la incidencia.

Software primera parte

Utilizaremos el siguiente software:

- *Lenguaje de programación C*
 - Compilador GCC C
 - Compilaremos a mano, usaremos el depurador desde entorno.
 - Visual Studio Code con las siguientes extensiones:
 - C/C++ IntelliSense, depuración y navegación de código
 - Paquete de extensiones C/C++

*Consulta las instrucciones de instalación en Discord e instala en tu ordenador **antes de este viernes :)**.*

Objetivos

- En tu primer año has aprendido cómo utilizar estructuras de datos preexistentes:
 - Lista (ArrayList, LinkedList)
 - Conjunto (HashSet, TreeSet)
 - Mapa (HashMap, TreeMap),...
- En este curso, estudiaremos cómo **diseñar, analizar e implementar eficientemente** y *desde cero* las estructuras de datos más importantes.

Entrevistas de trabajo para Internship

Google

Software Engineering Intern, MS, Summer 2027



Google



Mountain View, CA, USA; Austin, TX, USA; +28 more



Intern & Apprentice

Minimum qualifications

- Pursuing a Master's degree or post secondary or training experience with a focus on subjects in software development or other technical related fields.
- Experience in software development and coding in two or more general purpose programming languages (e.g., C, C++, Java, JavaScript, Python, etc.).
- Experience with data structures or algorithms gathered from inside or outside of school or work.

[Learn more](#)

<https://www.google.com/about/careers/applications/internships>



IBM Careers

Required technical and professional expertise

(Guidelines — transferable skills are encouraged)

- Coursework or project experience in software development using any programming language.
- Familiarity with fundamental concepts such as data structures, basic algorithms, or debugging.
- Exposure to front-end or back-end technologies (React, Node.js, Python, Java, SQL, etc.).
- Experience using Git or similar version control tools.
- Interest in cloud computing, containers, or modern deployment practices.
- Strong analytical thinking, problem-solving, and communication skills.
- Ability to collaborate and learn quickly in a team environment.

https://careers.ibm.com/en_US/careers/JobDetail/Software-Developer-Intern-2027/128497

<https://amazon.jobs/content/en/career-programs/university/sde>

Core competencies

During our internships, you'll build systems, design consumer products, and help with large applications. The skills listed below will help you excel.

Coding

A SDE's main job is writing code that can scale, is reliable, and is well-tested. You should be able to code by hand. Be ready to write real, working code. The goal is to write code that's almost ready for production. This is your chance to show off your coding skills.

Programming languages

Familiarity with a prominent language is a prerequisite for success. Be familiar with the syntax and nuances of at least one of the common languages, e.g. Java, JavaScript, Python, C#, C, C++, or Ruby.

NOTE: This list is not exhaustive. You have the ability to choose your preferred language throughout the process.

Data structures

Store and provide access to data in efficient ways. Understand the inner workings of common data structures and be able to compare and contrast their usage in various applications. Know the runtimes for common operations as well as how they use memory.

Algorithms

Object-oriented design

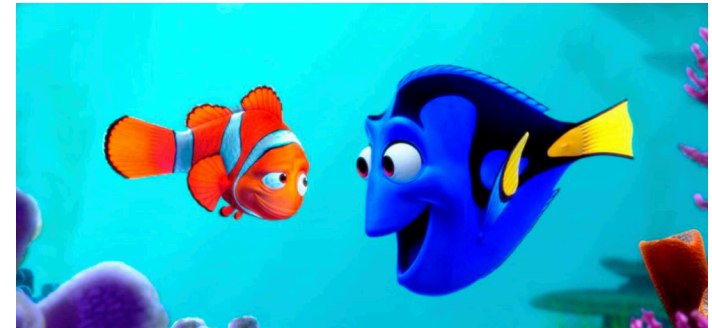
Tu cerebro no es un disco duro

- Mielinización y esfuerzo: Las conexiones neuronales duraderas solo se recubren de mielina cuando existe **fricción mental**.
- Filtro de prioridad: Si no aplicas tu conocimiento para resolver problemas, el cerebro asume que es irrelevante y lo borra en cuestión de días.



Si estudio de memoria con la IA

- Mielinización débil y delgada
- Circuitos aislados y rígidos
- Olvido rápido



Cuando te esfuerzas por resolver un problema...

- Mielinización densa y gruesa
- Grandes zonas de información
- Habilidades permanentes



Frustación

El impacto final en tu cerebro depende de la actitud que tomes cuando aparece esa incomodidad



Si abandonas ante la frustración

La ventana de plasticidad se cierra. Los oligodendrocitos no llegan a depositar la mielina porque el estímulo se detuvo.

El cerebro aprende que ante la dificultad, la solución es huir, reforzando la frustración como una barrera insalvable.

Si persistes e iteras tras el error

Se produce una mielinización masiva y acelerada. El cerebro experimenta el "sesgo de recompensa" cuando finalmente lo logras, liberando dopamina.

Esto no solo fija el conocimiento, sino que reconfigura tu mente para asociar el esfuerzo con el éxito (mentalidad de crecimiento).

Cómo y cuándo consulto a la IA

Cómo: Nunca preguntes por soluciones completas de código. Si te atrancas con un método, intenta primero preguntar para ver si entiendes bien que tiene que hacer el método. Si esto lo tienes claro, puedes preguntar que te guíe, pero que no te de la solución.

Cuándo: Debes trabajar de forma activa al menos 30 min antes de comenzar a preguntar.

Dos formas complementarias de enseñar

Instrucción explícita (paso a paso) 	Descubrimiento guiado (indagación) 
Idea: Yo modelo la solución, muestro el proceso y te guío con pasos claros.	Idea: tú construyes la solución con pistas, preguntas y andamiaje que te proporciono.
Harás: imitar el modelo → practicar con apoyo → resolver de forma autónoma (<i>Yo—Nosotros—Tú</i>).	Harás: pensar → probar → razonar (<i>Tú—Nosotros</i>).
Beneficio E.: reducir la carga cognitiva inicial para comprender conceptos difíciles.	Beneficio E.: explorar autónomo, generar pensamiento crítico y transferir lo aprendido.
Beneficio P.: acotado, poca incertidumbre.	Beneficio P.: diversidad, falta de aburrimiento.

¿Cómo trabajaremos en ED?

- Mini-lección explícita (modelado + ejemplo) → andamiaje (feedback vuestro mejora su calidad)
- Aprendizaje basado en problemas
- Puesta en común y defensa (pensar/razonar)

Sentimientos encontrados



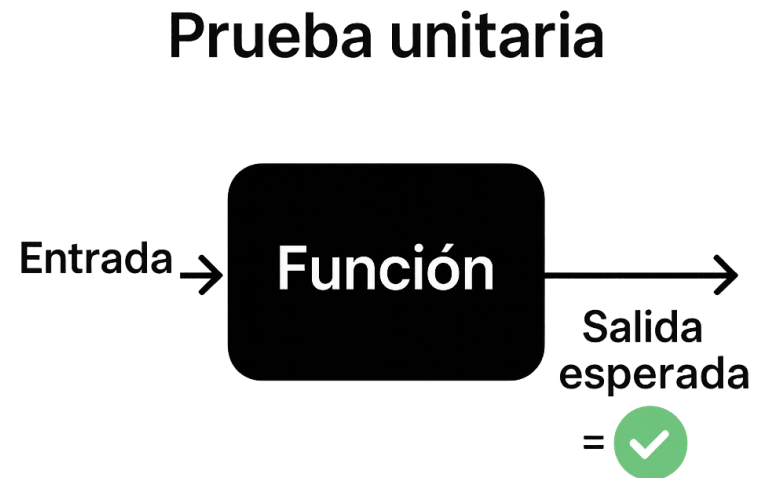
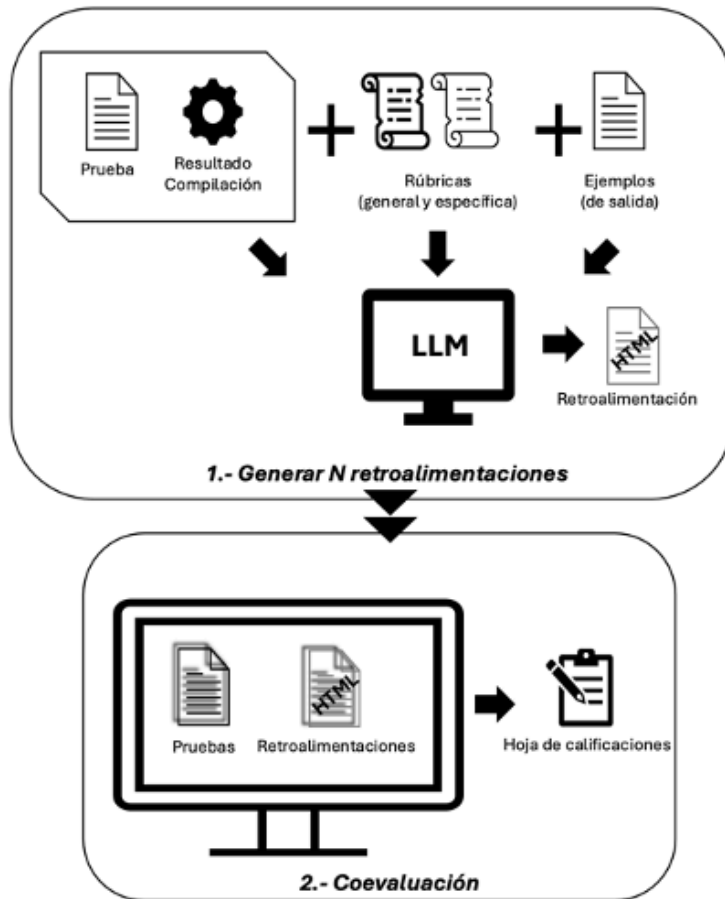
¿Cómo trabajaremos en ED con veteran@s?

- Cada semana se propondrá un examen o parte de un examen.
- En clase se trabajará ese examen, preguntando dudas al profesor.
- Deberás trabajar también en casa, de la forma propuesta anteriormente.

Ayudas basadas en IA

- Coevaluación
- Generador de ejercicios y soporte (estará la siguiente semana)

Coevaluación vs pruebas unitarias



Coevaluación, retroalimentación sobre calificación 1/3

Genera un toString que método mostrará la información de la cita como se muestra en el ejemplo, hay que tener en cuenta que si el idUsuario es distinto de null debe aparecer "Usuario 1" donde 1 será el idUsuario del usuario. En caso de que sea null esa parte no se mostrará.

Ejemplo sin usuario:

Neumologia: CHAT (faringe Moderada)

Neumologia: CHAT (faringe Leve)

Ejemplo con usuario:

Neumologia: Usuario 2 TELEFONICA (pulmones Leve)

Neumologia: Usuario 2 CHAT (faringe Moderada)

Coevaluación, retroalimentación sobre calificación 2/3

```
public String toString() {  
    StringBuilder sb = new StringBuilder("Neumologia: ");  
    if(this.getUsuario()!=null) {  
        sb.append("Usuario "+this.getUsuario().getId()+" ");  
    }  
  
    if(this.getTipoCita()==TipoCita.TELEFONICA) {  
        sb.append("TELEFONICA (");  
    }if(this.getTipoCita()==TipoCita.VIDEOLLAMADA) {  
        sb.append("VIDEOLLAMADA (");  
    }if (this.getTipoCita()==TipoCita.CHAT){  
        sb.append("CHAT (");  
    }  
  
    sb.append(this.afeccion + " " + this.gravedad +")");  
  
    return sb.toString();  
}
```

Neumologia: CHAT (faringe Moderada)
Neumologia: Usuario 2 TELEFONICA (pulmones Leve)

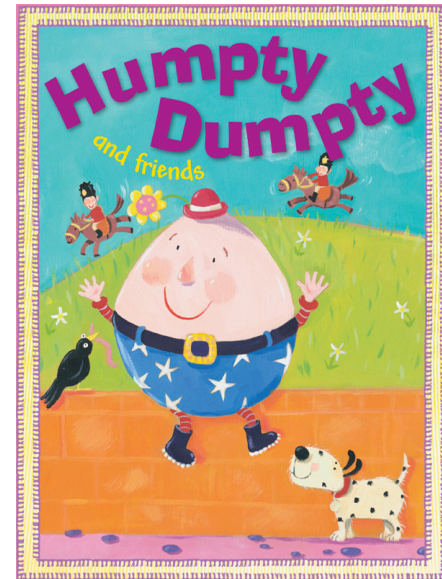
Coevaluación, retroalimentación sobre calificación 3/3

Forma de retroalimentar	Comentarios y nota
Prueba Unitaria	1/1
Prueba Unitaria como base	Todo genial en el toString()
Evaluación manual	Método toString (0.8/1): El enumerado se puede añadir al sb sin necesidad de hacer el triple if que has realizado, esto mejora la mantenibilidad.
Coevaluación	Método toString (0.8/1): El método está bien estructurado y utiliza <code>StringBuilder</code> en vez de concatenar <code>`String'</code> , lo que es más eficiente. El enumerado se puede añadir al sb sin necesidad de hacer el triple if que has realizado, simplemente <code>sb.append(this.getTipoCita())</code> . Esto mejora la claridad del código y la mantenibilidad. Hace que sea más fácil de extender (otro valor) y que sea menos propenso a errores.

Un ejemplo de importancia (I)

- Dado un texto, encuentra la palabra más frecuente en él

Humpty Dumpty se sentó en un muro,
Humpty Dumpty tuvo una gran caída.
Todos los caballos del rey y todos los
hombres del rey
No pudieron volver a juntar a Humpty.



Un ejemplo de importancia (II)

- Utilizaremos un *par* para almacenar una palabra (`String`) y su frecuencia (`Integer`).
- Una *estructura de datos* almacenará todos los pares.
- Este será el estado de la estructura de datos después de procesar los dos primeros versos del texto de ejemplo:

(Humpty, 2)

(Dumpty, 2)

(se, 1)

(sentó, 1)

(en, 1)

(un, 2)

(muro, 1)

(tuvo, 1)

(una, 1)

(gran, 1)

(caída, 1)

Interfaz para la Estructura de Datos

- Consideremos una estructura de datos genérica que almacena pares de claves y valores.
- En nuestro caso, la clave es un `String` (palabra) y el valor es un `Integer` (frecuencia).
- Las operaciones que esta estructura de datos debe admitir son:

```
// inserta un par clave-valor en la estructura de datos
public void put (Key key, Value value);

// obtiene el valor asociado a una clave o null si la clave no está en la estructura de datos
public Value get (Key key);

// verifica si la estructura de datos contiene una clave
boolean contains(Key key);

// devuelve todas las claves en la estructura de datos
public Iterable<Key> keys();
```

Algoritmo

- El algoritmo para encontrar la palabra más frecuente en un texto es:

```
// crea una estructura de datos vacía
Pares<String,Integer> pares = new Pares<>()

// procesa el texto
para cada palabra en el texto {
    Integer n = pares.get(palabra)
    si(n == null)
        pares.put(palabra, 1) // primera aparición de la palabra
    sino
        pares.put(palabra, n+1) // incrementa el contador para la palabra
}

// encuentra el contador más grande
Integer maxOcurrencias = 0
para cada palabra en pares.keys() {
    Integer n = pares.get(palabra)
    si(n > maxOcurrencias)
        maxOcurrencias = n
}
```

Diferentes Estructuras de Datos

- Ejecutaremos **el mismo algoritmo** utilizando diferentes estructuras de datos:
 - *Lista enlazada secuencial*
 - *Array ordenado utilizando búsqueda binaria*
 - *Árbol de búsqueda binaria balanceado*
- Compararemos el rendimiento de las diferentes estructuras de datos para resolver el problema.

¡Prueba en tu maquina!

Complejidad Computacional

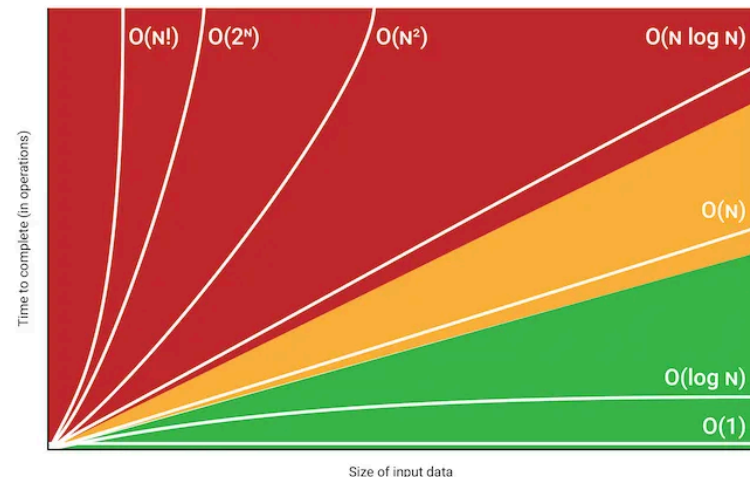
- Un programa debe ser *correcto*: debe producir la salida esperada.
- Un programa también debe ser *eficiente*: debe producir la salida esperada lo más rápido posible.

Complejidad Computacional (II)

- Para calcular su salida, un programa debe realizar un *número de operaciones*. Este número es una buena estimación de su *tiempo de ejecución*.
- El número de operaciones que realiza un programa ¡NO ES FIJO! Depende de la cantidad de datos que procesa (*tamaño del problema*). Denotaremos este tamaño como n .
- La *complejidad computacional* de un programa es una *función* de su tamaño del problema n que devuelve el número de operaciones que el programa realiza para dicho tamaño del problema.

Complejidad Computacional (III)

- Es **difícil** contar el número de operaciones *exacto*.
- La **notación big O** proporciona un *límite superior* para el número de operaciones que realiza un programa en tamaños grandes.
- Hay diferentes *clases de complejidad computacional*.



Complejidad de las Operaciones para Diferentes Estructuras de Datos

La complejidad de las operaciones (dónde n es el número de pares almacenados) es:

Estructura de Datos	put	get	contains	keys
Lista enlazada secuencial	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Array ordenado utilizando búsqueda binaria	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Árbol de búsqueda binaria balanceado	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$

Contador de palabras implementado en Java

Tiempos de Ejecución en el contador de palabras

- M1 pro, VSCode, OSX Sonoma 14.2.1, 16GB

tale.txt (135 635 palabras)	Tiempo	vs. Lista	vs. Array
Lista Enlazada	5.11 segundos		
Array	0.28 segundos	18 veces más rápido	
Árbol	0.14 segundos	37 veces más rápido	2 veces más rápido

Programa de Estudio

- El Lenguaje de Programación C
- Estructuras de Datos en C
- Repaso del Lenguaje de Programación Java
- Tipos de Datos Abstractos y Estructuras de Datos Lineales
- Árboles
- Tablas Hash
- Grafos

Sistemas de Evaluación

Bloques Temáticos y Pesos en la Nota Final:

- **Bloque 1:** El Lenguaje de Programación C y Estructuras de Datos en C (33.33%)
- **Bloque 2:** Lenguaje de Programación Java, Tipos de Datos Abstractos, Estructuras de Datos Lineales, Árboles, Tablas Hash y Grafos (66.66%)

Exámenes:

- **Tres pruebas durante el Curso:**
 - Uno para el Bloque 1: C
 - Dos para el Bloque 2: Árboles y Tablas Hash/Grafos en Java

Sistemas de Evaluación (II)

Criterios de Aprobación:

- Una nota de **2 puntos o más** (sobre 10) en cualquier bloque se considera una *aprobación para ese bloque*.
 - Si un estudiante repite un bloque aprobado, se utilizará la última nota, incluso si es más baja.
- Para **aprobar el curso**, los estudiantes deben:
 - Aprobar cada bloque y
 - Obtener una nota final ponderada de al menos **5 puntos sobre 10** (según los pesos mencionados anteriormente)

Para obtener más detalles, por favor consulta la [guía docente](#).

Bibliografía

- Sedgewick, R., & Wayne, K. (2011). Algorithms. Addison-Wesley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2010). Introduction to Algorithms. McGraw-Hill.
- Goodrich, M. T., & Tamassia, R. (2006). Data Structures & Algorithms in Java. John Wiley & Sons.
- Kernighan, B. W., & Ritchie, D. M. (2002). The C Programming Language. Prentice Hall.
- King, K. N. (2008). C Programming: A Modern Approach (2nd ed.).